

Panduan Lengkap — Pembedahan 3 Dokumen

Tugas Besar Big Data · Credit-Card Default (Taiwan 2005)

Penjelasan mendalam tiap bagian dari: `HOSTING_ORACLE`, `PANDUAN_BELAJAR_GITHUB_HOSTING`, `PANDUAN_BELAJAR_TECH_STACK`.

Dokumen ini membedah **maksud, contoh, dan kesalahan umum** tiap bagian dari ketiga dokumen tersebut — bukan sekadar mengulang isinya, tetapi menjelaskan "kenapa" dan "bagaimana" lebih dalam.

A. HOSTING_ORACLE — pembedahan tiap bagian

Dokumen ini = resep menaruh stack Anda di server cloud gratis **dengan aman**. Urutannya sengaja: amankan dulu, baru jalankan.

A.0 Security Model (paling atas)

- **Isi:** menegaskan layanan (Hive/Spark/Kafka) **tanpa autentikasi & plaintext**, lalu menetapkan 4 lapis pertahanan.
- **Maksud mendalam:** "plaintext" = data lewat jaringan tanpa enkripsi (bisa disadap); "tanpa autentikasi" = tidak ada cek username/password, siapa pun yang menyentuh port dapat akses penuh. Pada Hive (bisa DROP) & Spark (bisa eksekusi kode), ini setara **Remote Code Execution**.
- **Kenapa di paling atas:** agar Anda paham risikonya **sebelum** membuka port (sikap secure-by-default).
- **Kesalahan umum:** menganggap "cuma tugas, tak apa dibuka" — bot pemindai menemukan port terbuka dalam hitungan menit.

A.1 Provision VM (buat server)

- **Isi:** akun Oracle, shape `VM.Standard.A1.Flex` 4 OCPU/24 GB, Ubuntu 24.04 ARM, boot volume ~120 GB.
- **Maksud mendalam:** **OCPU** = unit core Oracle; **24 GB RAM** krusial karena JVM Hive + Spark rakus memori; **boot 120 GB** karena image Docker besar (default 50 GB cepat penuh). 200 GB pertama gratis.
- **Kesalahan umum:** memilih shape AMD micro (1 GB) yang juga gratis → stack tak jalan.
- **Catatan realistis:** kapasitas ARM sering "Out of capacity" → ulangi / ganti region.

A.2 Open the Network (DUA lapis firewall)

- **Isi:** mode aman hanya buka **port 22 (SSH)**; dua tempat: (2a) **VCN Security List** (firewall cloud) + (2b) **ufw** (firewall di dalam VM).
- **Maksud mendalam:** sumber 90% masalah "tidak bisa connect". Paket harus lolos **dua** pintu: `Internet -> [VCN Security List] -> [ufw di VM] -> layanan`. Buka satu tapi lupa lainnya = tetap gagal. `ufw limit 22/tcp` (bukan `allow`) otomatis mem-blok IP yang SSH terlalu sering (anti brute-force).
- **Kesalahan umum:** hanya atur Security List, lupa `ufw` masih `REJECT` default di Ubuntu Oracle.

A.3 Install Docker

- **Isi:** Docker Engine + plugin compose (arm64), tambah user ke grup docker.
- **Maksud mendalam:** `usermod -aG docker $USER` agar bisa docker tanpa sudo; **wajib** `logout/login` agar grup aktif.
- **Kesalahan umum:** lupa relogin → `permission denied` saat `docker ps`.

A.4 Ship the Project (kirim kode)

- **Isi:** `git clone` dari GitHub Anda (atau scp); pastikan input ikut.
- **Maksud mendalam:** di sinilah **GitHub & hosting bertemu** — GitHub jadi jembatan kode laptop → server. Tapi `.gitignore` mengabaikan `raw/*`, jadi file di `raw/` **tidak** ikut clone. Solusi: andalkan `synthetic/synthetic_credit_clients.csv` (ikut), scp file raw terpisah, atau jalankan `extract.py` di VM untuk FRED.
- **Kesalahan umum:** clone lalu heran data tak ada — karena memang di-ignore.

A.5 Configure .env

- **Isi:** `cp .env.example .env` lalu isi `FRED_API_KEY` & `PUBLIC_IP`.
- **Maksud mendalam:** `.env` berisi rahasia & **tidak** ikut git (sengaja), jadi di server baru harus dibuat manual. `PUBLIC_IP` hanya untuk mode insecure (advertised listener Kafka).
- **Kesalahan umum:** lupa `chmod 600 .env` → file rahasia bisa dibaca user lain.

A.6 Build & Run (mode aman)

- **Isi:** `docker compose build` lalu `up -d` (**tanpa** `prod.yml`); verifikasi metastore + `chmod` volume.
- **Maksud mendalam:** file base saja = semua port di `127.0.0.1` (aman). Menambah `-f docker-compose.prod.yml` baru membuka publik (insecure). `up -d` = background (detached).
- **Kesalahan umum:** ikut menambah `prod.yml` padahal ingin mode aman.

A.7 Populate the Warehouse (jalankan pipeline)

- **Isi:** `extract` → `transform` → `load` → ELT → KPI; CTGAN **di-skip**; tahap tulis-warehouse pakai `-u root`.
- **Maksud mendalam:** `-u root` mengatasi bentrok kepemilikan file antara user Hive (uid 1000) & Spark (uid 1001) pada volume bersama. CTGAN di-skip karena berat (PyTorch) & hasilnya sudah ada.
- **Kesalahan umum:** menjalankan CTGAN bareng Hive → RAM habis → Hive OOM (exit 137).

A.8 Secure Access via SSH Tunnel (cara akses)

- **Isi:** satu perintah `ssh -N -L ...` meneruskan banyak port (10000, 10002, 8080, 8085) ke laptop; dashboard konek ke `localhost`.
- **Maksud mendalam** — bedah `-L 10000:localhost:10000 ubuntu@<IP>`:
- `10000` (pertama) = port di **laptop Anda**;
- `localhost:10000` = tujuan **dilihat dari sisi VM**;
- artinya "apa pun yang saya kirim ke `localhost:10000` di laptop, teruskan terenkripsi ke `localhost:10000` di VM". `-N` = tanpa shell, cuma tunnel. DSN ODBC tetap `localhost` (bukan IP).
- **Kesalahan umum:** mengisi DSN dengan IP publik VM → gagal, karena port tak terbuka publik di mode aman.

A.9 Harden the VM

- **Isi:** SSH kunci-saja, fail2ban, unattended-upgrades, `chmod 600 .env`, opsi reverse-proxy Caddy + opsi HiveServer2 LDAP.
- **Maksud mendalam:** lapis ke-4. Mematikan login password menutup serangan tebak-password; fail2ban mem-ban IP nakal; Caddy beri HTTPS + Basic Auth bila butuh UI browser tanpa tunnel.
- **Kesalahan umum:** mematikan `PasswordAuthentication` **sebelum** memastikan kunci SSH berfungsi → terkunci dari VM sendiri.

A.10 Fallback QEMU & Teardown

- **Isi:** jika build Spark ARM bermasalah, jalankan image amd64 via emulasi QEMU; setelah selesai `down -v + terminate instance`.
- **Maksud mendalam:** QEMU mensimulasikan CPU amd64 di atas ARM (lambat tapi jalan) — jaring pengaman. **Teardown wajib** secara keamanan: server menganggur dengan layanan no-auth = risiko; terminate juga mengosongkan kuota.

B. PANDUAN_BELAJAR_GITHUB_HOSTING — pembedahan tiap bagian

Dokumen **belajar** (konsep), bukan langkah operasional. Bagian hosting-nya ringkasan dari HOSTING_ORACLE.

B.1.1–1.2 Git vs GitHub & konsep inti

- **Maksud mendalam:** Git **lokal & offline**, GitHub **online**. Tabel istilah (repo/commit/branch/remote/staging) = kosakata wajib; tanpa paham **staging area**, `git add` terasa misterius.
- **Analogi:** commit = "checkpoint game"; branch = "save slot berbeda"; remote = "upload save ke cloud".

B.1.3 Siklus kerja Git

- **Maksud mendalam:** `add` → `commit` → `push` adalah jantung Git. Tiga ruang: *working directory* → *staging* → *repository*; push memindahkan riwayat ke GitHub.
- **Kesalahan umum:** mengira `git commit` langsung ke GitHub — tidak; commit hanya lokal sampai push.

B.1.4–1.5 .gitignore, keamanan, artefak besar

- **Maksud mendalam:** dua keputusan: (1) rahasia (`.env`) tak boleh ke Git — kebocoran kunci nyata berbahaya; (2) file besar/hasil-generate (warehouse/staging/) diabaikan (Git buruk untuk biner besar; file bisa dibuat ulang). CSV sintetis disimpan karena ia **input**.
- **Kesalahan umum:** terlanjur `git add .env` sebelum ignore → `git rm --cached .env`.

B.1.6 Autentikasi

- **Maksud mendalam:** GitHub menolak password biasa sejak 2021. **PAT** = token izin (scope repo), ibarat "password khusus aplikasi". **Credential Manager** (bawaan Git Windows) paling mudah — login browser sekali, tersimpan. **SSH key** untuk yang mahir.
- **Kesalahan umum:** memasukkan password akun → selalu ditolak; harus PAT.

B.1.7–1.8 Langkah push & troubleshooting

- **Maksud mendalam:** urutan `init` → `branch -M main` → `add` → `status` → `commit` → `remote add` → `push -u`. `git status` = **checkpoint keamanan** sebelum commit (pastikan `.env` tak muncul). Tabel troubleshooting menutup kasus tersering (rejected → `pull --rebase`).

B.2.1–2.4 Konsep hosting & Docker

- **Maksud mendalam:** GitHub simpan kode statis vs **hosting** jalankan program hidup. Docker: image (cetakan) vs container (instance) vs volume (data permanen). Wajib paham sebelum cloud.

B.2.5 Model keamanan berlapis

- **Maksud mendalam:** versi ringkas 4 lapis (localhost binding → firewall SSH-only → SSH tunnel → hardening). **Bagian bernilai tinggi untuk penilaian** (menunjukkan kesadaran keamanan).

B.2.6–2.11 Mode, alur, koneksi, biaya, troubleshooting

- **Maksud mendalam:** tabel **mode aman vs terbuka** (perintah & risiko), alur 11 langkah end-to-end, penegasan **GitHub↔hosting** lewat `git clone` di VM, plus catatan kuota & teardown.
-

C. PANDUAN_BELAJAR_TECH_STACK — pembedahan tiap bagian

Format tiap teknologi: **Apa** · **Kenapa di sini** · **Konsep kunci** · **Peran (file)** · **Cara belajar**.

C.0 Gambaran besar

Alur: Sumber → Python → Kafka → (ETL Spark / ELT Hive) → Hive(+Postgres, Parquet) → Tableau/Power BI, dibungkus Docker. Memahami **aliran** ini membuat peran tiap komponen masuk akal.

C.1 Python

Dipilih karena ekosistem data terkaya; `requirements.txt` mengunci versi agar reproduibel. Semua script proyek = Python.

C.2 Kafka (KRaft)

"Kantor pos" — producer kirim ke topic, consumer ambil. **KRaft** = tanpa Zookeeper (lebih simpel).

Peran: memisahkan extract dari pemrosesan (idempoten, bisa di-stream). File: `kafka_producer/consumer.py`.

C.3 Spark / PySpark

Engine data terdistribusi; **DataFrame** + **lazy evaluation** (jalan saat ada *action*). `local[*]` = semua core 1 mesin. Inti ETL: `etl_pipeline/transform.py` (clean, unpivot, 5 fitur).

C.4 Hive

Menjadikan file Parquet bisa di-SQL; **Metastore** (katalog) terpisah dari **HiveServer2** (endpoint ODBC). Tempat transform **ELT** (window/OLAP). File: `warehouse/*.sql`, `etl_pipeline/transform.sql`.

C.5 PostgreSQL

"Buku katalog" Hive — menyimpan metadata tabel; butuh driver JDBC; tidak diekspos publik (aman). Tak perlu didalami untuk proyek ini.

C.6 Parquet

Format **kolumnar** + kompresi → cepat & hemat untuk analitik vs CSV. Semua tabel warehouse memakainya agar terbaca ODBC.

C.7 CTGAN (SDV)

GAN (generator vs discriminator) untuk data **tabel**; memperbanyak 30k → 150k, *conditional sampling* menjaga rasio default 22%. **Wajib catat bias** (fidelitas ≤ sumber). File: `synthetic/train_ctgan.py`, `CTGAN_METHOD.md`.

C.8 FRED API + UCI

Memenuhi multi-source & multi-format. FRED = REST API JSON dengan API key (rahasia di `.env`). File: `etl_pipeline/extract.py`.

C.9 Docker & Compose

Container = aplikasi terisolasi yang jalan sama di mana saja; Compose merangkai 7 layanan. Konsep multi-arch (amd64 vs ARM) memicu penggantian image Spark.

C.10 Tableau & Power BI

Tableau→ETL, Power BI→ELT, agar dua pipeline dibandingkan visual. Konek via **ODBC** ke HiveServer2 (atau CSV fallback). File: `dashboard/DASHBOARD_BUILD_GUIDE.md`.

C.11–C.13 ETL vs ELT, glosarium, urutan belajar

ETL olah sebelum load (PySpark, fitur ML); **ELT** olah di dalam warehouse (SQL window/OLAP); proyek pakai keduanya untuk dibandingkan (ELT ~5× lebih cepat di sini). Glosarium + urutan belajar fondasi→atas (Python→SQL→Docker→Spark→Hive→Kafka→CTGAN→BI→hosting).

Cara memakai ketiga dokumen ini bersama

Mau PAHAM teknologinya? -> PANDUAN_BELAJAR_TECH_STACK Mau PAHAM Git & konsep hosting? -> PANDUAN_BELAJAR_GITHUB_HOSTING Mau PRAKTIK hosting nyata? -> HOSTING_ORACLE (langkah operasional)

Alur belajar disarankan: TECH_STACK (paham komponen) → GITHUB_HOSTING (paham Git + konsep deploy) → HOSTING_ORACLE (eksekusi di server).